

CHILLISOURCE GAME ENGINE PARTICLE STUDY

By

Angela Nicole Gross

Bachelor of Science, The University of Montana, Missoula, MT, 2014

Thesis

presented in partial fulfillment of the requirements
for the degree of

Master of Science
in Computer Science

The University of Montana
Missoula, MT

Autumn 2016

Approved by:

J.B. Alexander S. Ross Ph.D., Dean
Graduate School

Travis Wheeler Ph.D., Chair
Computer Science

Michael Cassens M.S.
Computer Science

Johnathan Bardsley Ph.D.
Mathematical Sciences

© COPYRIGHT

by

Angela Nicole Gross

2016

All Rights Reserved

Gross, Angela N., M.S., December 2016

Computer Science

ChilliSource Game Engine Particle Study

Chairperson: Travis Wheeler

This is the short summary of why this thesis is awesome and *YOU* really want to read it!

ACKNOWLEDGMENTS

Thanks guys ...

TABLE OF CONTENTS

COPYRIGHT	ii
ABSTRACT	iii
ACKNOWLEDGMENTS	iv
CODE LISTINGS	vii
LIST OF FIGURES	viii
LIST OF TABLES	ix
CHAPTER 1 INTRODUCTION	1
1.1 Thesis Organization	1
CHAPTER 2 BACKGROUND	2
2.1 Particle Systems	2
2.1.1 Overview	2
2.1.2 Particle Model	3
2.1.2.1 Life Cycle	3
2.1.2.2 Particle Attributes	4
2.1.2.3 Hierarchy	5
2.2 The ChilliSource Game Engine	5
2.2.1 Overview	5
2.2.2 Architecture	6
2.2.3 Life Cycle	7

2.3	ChilliSource Particle Effect Components	10
2.3.1	Overview	10
2.3.2	Usage	10
2.3.3	Architecture	12
2.3.4	Attributes	12
2.3.5	Life Cycle	12
2.3.5.1	Overview	12
2.3.5.2	Update Event	12
2.3.5.3	Render Event	12
CHAPTER 3	THE CHILLISOURCE AUTOMATED PONG GAME . .	13
CHAPTER 4	CHILLISOURCE PARTICLE EFFECT COMPONENT	
	CASE STUDIES	14
CHAPTER 5	CHILLISOURCE PARTICLE EFFECT COMPONENT	
	OBSERVATIONS	15
BIBLIOGRAPHY		16

CODE LISTINGS

2.1	Creating a basic particle effect component with ChilliSource.	10
2.2	Creating a scene that will use the basic particle effect component with ChilliSource.	11

LIST OF FIGURES

Figure 2.1	Particle effect created by William T. Reeves	2
Figure 2.2	Cubes emitting particles	3
Figure 2.3	Basic structure of a ChilliSource application	7
Figure 2.4	An example of the Update life cycle event	9

LIST OF TABLES

CHAPTER 1 INTRODUCTION

1.1 Thesis Organization

The rest of this thesis is organized as follows:

- **Chapter 2** Review of particle systems and the ChilliSource game engine
- **Chapter 3** Overview of the automated pong game that was created to facilitate case studies
- **Chapter 4** Discussion of the two case studies performed to explore contention in large data structures
- **Chapter 5** Discussion of three interesting aspects of the ChilliSource particle effect component

CHAPTER 2 BACKGROUND

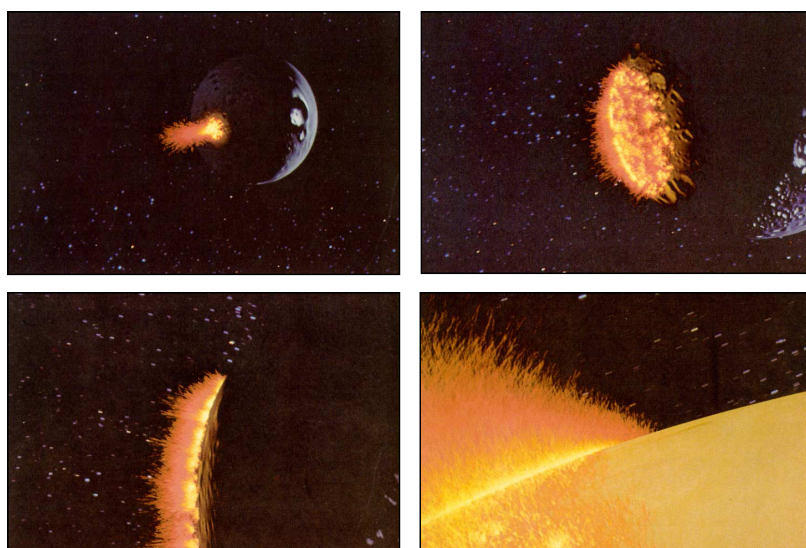


Figure 2.1: The particle effect created by William T. Reeves from the film *Star Trek II: The Wrath of Khan*

2.1 Particle Systems

2.1.1 Overview

Visual representation of "fuzzy objects", or objects that do not have well-defined surfaces, in computer graphics is generally considered a difficult task due to the non-deterministic changes in the fuzzy object's shape and form. When working on the film *Star Trek II: The Wrath of Khan*, a researcher at Lucasfilm Ltd., William T. Reeves, developed what he called a **particle system**, which is "... a collection of many minute particles that together represent a fuzzy object. Over a period of time, particles are generated into a system, move

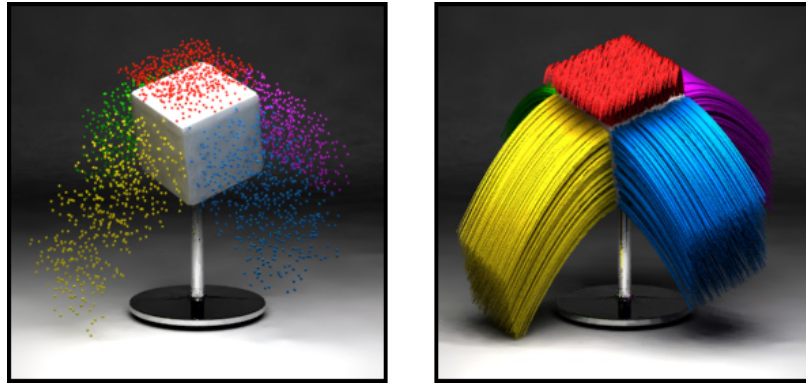


Figure 2.2: Left: A cube emitting a stream of 1000 animated particles from 5 of its 6 faces. Right: A cube emitting a stream of 1000 static particles, or strands, from 5 of its 6 faces

and change from within the system, and die from the system.” [1] Although the concept of modeling these objects as a collection of particles was not novel, Reeves' basic framework for particle systems is still widely used today to create complex visual effects in video games and films.

Fuzzy objects like sparks, clouds, stars, water, and crowds of people can be easily translated and modeled as a particle system, but particle systems can also be used to model objects that contain numerous strands like hair and grass. These objects can be represented by rendering every strand as the entire lifetime of a particle in the system and is then treated as a single strand. The particles in a system that have a distributed lifetime (e.g. sparks) are thought of as *animated* and particles that are rendered all at once (e.g. hair) are thought of as *static*. For the duration of our own study, we have exclusively focused on animated particles.

2.1.2 Particle Model

2.1.2.1 Life Cycle

A particle system has 4 main events that defines its life cycle:

1. Emission (or Generation, according to Reeves)

2. Update
3. Render
4. Death (or Extinction, according to Reeves)

The **Emission** event is the initial process in which particles are generated with their initial values. Often a "generation shape"[1] is used during the emission phase that defines the two or three dimensional space that the particles can originate, or emit, from. Emission can be increased or decreased in frequency using the *emission rate*, and the number of particles that enter the particle system each emission can also be changed using the *particles per emission* variable. After emission, particles in the particle system undergo the **Update** and **Render** events. These events are repeated in perpetuum until the particle dies, or undergoes the **Extinction** phase. The **Update** phase involves iterating over all particles throughout the system and updating their attributes (e.g. position, velocity, etc.). The **Render** phase, on the other hand, is the phase in which all of the particles are actually drawn to the screen.

2.1.2.2 Particle Attributes

As mentioned above, each individual particle in a particle system will have attributes that change over time. Although every particle system is different, the following is what Reeves considered to be the core attributes of a particle:

- Position
- Velocity
- Size
- Color
- Transparency
- Shape
- Lifetime

A particle has a position in two or three dimensional space, and its initial position is determined by the emission process. Other attributes that may also change over time

include its speed, direction, size, color, and transparency. Its lifetime, however, is pre-determined at emission, and it is the attribute that dictates how long the particle "lives", or remains on the screen. In order to decrease uniformity and increase realism, all of these values can vary across the particles in the system. For example, the particles in a given particle system could have a varying lifetime of 5 to 10 seconds so that the particles that shared an emission would not all die simultaneously.

2.1.2.3 Hierarchy

It is often convenient to design a particle system such that it is made up of a hierarchy of child particle systems. This hierarchy "can be used to exert global control on a complicated fuzzy object that is composed of many particle systems." [1] In other words, a fuzzy object could easily be transformed in overall shape, size, color, etc. by sweeping through the descendant particle systems. It also allows the grouping together of particle systems that have different types of particles. For example, a campfire fuzzy object would need smoke, sparks, and the flame itself. All of these components would have distinct particle definitions and would be separate particle systems. A particle system hierarchy, in this case, would allow the smoke, sparks, and flame particle systems to be bundled together as a single fuzzy object.

2.2 The ChilliSource Game Engine

2.2.1 Overview

Chosen for its open source and well-thought out architecture, ChilliSource is the game engine that was used in our study of particle systems. ChilliSource is a free, open source game engine created by ChilliWorks which is a group that is part of United Kingdom's Tag Games. It allows developers to create 2D and 3D content for iOS, Android, Kindle, and Windows operating systems in C++ 11. Some of its key features include networking, shader support, responsive (i.e. adaptive to different resolutions) graphical user interfaces, modular

and extensible lighting and shadows, skinned animation, Facebook connect integration, consistent API, and automatic builds of iOS/Android/Kindle applications. [2]

2.2.2 Architecture

The structure of a ChilliSource application is primarily composition based in that it relies on a hierarchy of modules to define its functionality. The main types of modules that comprise the application are *States*, *Systems*, *Entities* and *Components*. Figure 2.3 demonstrates the architecture of a ChilliSource application using these basic modules.

The *Application* instance initializes and manages *Application Systems*, which are modules that describe functionality used across the entire application. A non-exhaustive list of some application systems include the *Task Manager*, *Texture Provider*, *Render System*, and the *State Manager*. The *State Manager*, as shown in Figure 2.3, manages a stack of *States* and ensures that the state on top of the stack is active.

States can be thought of as different parts of the application such as a main menu, inventory, settings menu, pause screen, and the game scene itself. *States*, like the *Application*, initialize and manage *State Systems*. *State Systems* are similar to *Application Systems* except they describe functionality that only pertains to a single state and its lifetime is limited to that state. An important state system, the *Scene*, manage *Entities* that form that scene. The base *State* also creates and manages the *Canvas*, *Gesture*, and "User" state systems. The *Canvas* renders the user interface, the *Gesture* state system helps receive input events like pinching or tapping, and "User" state systems are custom modules built by the developer to describe functionality for the state.

Entities are essentially objects within the game that are defined by both (a) a transform describing their rotation, size, and position and (b) *Components*, which are modules that describe the entity's functionality. *Entities* can also contain a hierarchy of entities in which, much like in particle systems, their transforms and life cycle events are linked by the hierarchy. Some examples of *Components* are AI Controllers, Health, Damage, and Physics.

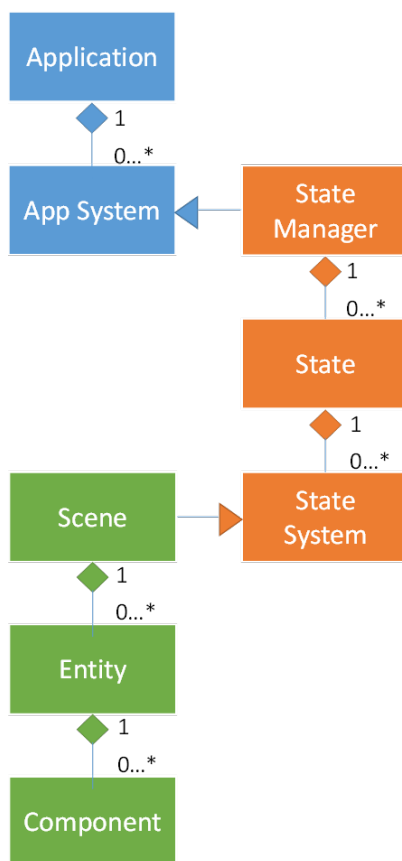


Figure 2.3: The basic structure of a ChilliSource application

ChilliSource's architecture favors composition over inheritance. For example, if we wanted to create enemy and player *Entities*, we may be tempted to create a base Human *Entity* class that had health related methods and attributes. In order to truly adhere to the spirit of ChilliSource's design, however, you would create a *Component* called something like **HealthComponent** that could be attached to any single *Entity*. In this way, ChilliSource lends itself to a great deal of code reusability and flexibility.

2.2.3 Life Cycle

The majority of game engines follow a life cycle very similar to the one for particle systems that Reeves' developed, which is:

1. Initialize
2. Update
3. Draw
4. Stop

The **Initialize** step usually involves loading textures, sounds, and anything else that the engine needs to do before starting the main game loop. The **Update** step is the heart of game, and is usually driven by some kind of state machine. Some examples of typical states include Menu, Game, Settings, and Exit. The **Draw** step is also part of the main game loop and, just as it sounds, this is where everything is drawn to the screen. Typically, there is some kind of renderer that handles all of the low-level calls to the graphics card (e.g. OpenGL, DirectX, etc.). And, optionally, there is the **Stop** step in which the game has been stopped and all assets and resources will be released. This may or may not be included as a separate state in the **Update** phase, but it is good to logically separate the two events.

ChilliSource, as you may expect, adheres to the basic "Initialize, Update, Draw, and Stop" phases. However, there are a great deal of other events that are perpetuated from the *Application* instance. The following is a list of the main life cycle events that all modules receive:

1. Init
 - This initializes the application and begins the **Update** event.
2. Update
 - This updates the application, its systems, and the state manager's states.
3. Render
 - This renders objects and user interface elements to the screen and is the main draw loop of the application.
4. Suspend

- This will notify active state(s) to pause and it will suspend all systems in reverse order.
5. Resume
 - This resumes the application from a suspended state.
 6. Background
 - This occurs when the application is not in view any longer.
 7. Foreground
 - This occurs when the application is in view again.
 8. Destroy
 - This cleans up the application and releases all systems in reverse order.

All of these events are kicked off from the *Application* instance. The flow of events looks something like *Application* → *Application Systems (State Manager)* → *States* → *State Systems (Scene)* → *Entities* → *Components*. Figure 2.4 shows what this flow looks like for the **Update** life cycle event using psuedocode. Note, however, that each event means something slightly different depending on which module it is called in (e.g. **Update**

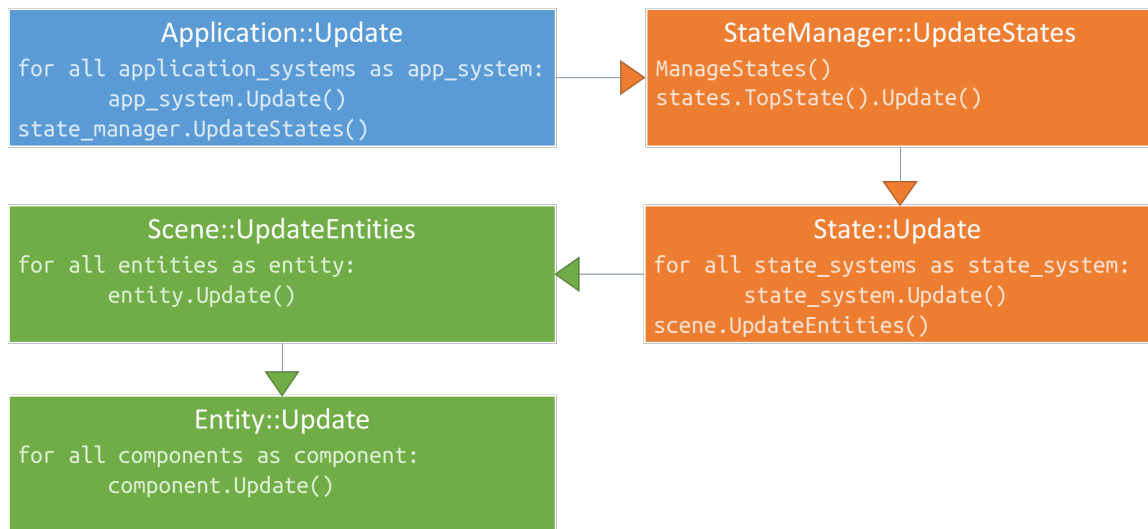


Figure 2.4: An example of the **Update** life cycle event being perpetuated from the *Application* all the way down to a *Component*

in the *Application* means something different than the **Update** in a *State*). Additionally, there are a number of other events that are not discussed here that only have meaning for specific modules (e.g. **OnAddedToEntity** for the *Component* module). See [3] for more information.

Although these are the main events that are automatically pushed by the application, the game developer can create custom events that can be subscribed to by entities, components, and systems. An *Event* can open, close, and notify *Connections*, where a *Connection* allows its user to listen for *Events*. Some examples of custom events are entity collisions, player death, cut scene trigger, game lost, game won, etc.

2.3 ChilliSource Particle Effect Components

2.3.1 Overview

A special type of component in ChilliSource is the *Particle Effect Component*. It inherits from the *Render Component* class¹, which is, just as it sounds, a component that is rendered to the screen. *Render Components* have methods for visibility, culling, casting shadows, and usage of its transformation matrix and sort predicates. They also have virtual methods that child classes can define for materials, transparency, rendering, and bounding boxes. The exact details of how a *Render Component* works is not in the scope of this paper, but it is worth knowing that the *Particle Effect Component* is not only updated by its parent entity but is also rendered by the *Application*'s renderer.

2.3.2 Usage

Hello

¹It's important to note here that the version of ChilliSource that I'm referring to is 1.6.0 and it is currently at version 2.0.0. In version 2.0.0, the rendering system is completely overhauled and *Particle Effect Components* are *Volume Components* and not *Render Components*. This new system was not studied and will not be discussed in depth here.

Code Listing 2.1: Creating a basic particle effect component with ChilliSource.

```

1 CS::EntityUPtr State::CreateParticleEffect()
2 {
3     // Retrieve the singleton application instance through a static method
4     auto app = CS::Application::Get();
5
6     // Retrieve the resource pool application system to create a particle effect
7     // from a JSON csparticle file
8     auto resourcePool = app->GetResourcePool();
9     auto particleEffect = resourcePool
10         ->LoadResource<CS::ParticleEffect>
11         (CS::StorageLocation::k_package,
12          "Particle/generic.csparticle");
13
14     // Using a render component factory application system and the particle
15     // effect, create a particle effect component
16     auto renderComponentFactory = app->GetSystem<CS::RenderComponentFactory>();
17     auto particleComponent = renderComponentFactory
18         ->CreateParticleEffectComponent(particleEffect);
19
20     // Set the particle component's playback to looping
21     particleComponent
22         ->SetPlaybackType(CS::ParticleEffectComponent::PlaybackType::k_looping);
23
24     // Create an entity, and add the particle component to the entity
25     auto particleEntity = CS::Entity::Create();
26     particleEntity->AddComponent(std::move(particleComponent));
27
28     return particleEntity;
29 }

```

Code Listing 2.2: Creating a scene that will use the basic particle effect component with ChilliSource.

```

1 void State::OnInit()
2 {
3     // Retrieve the resource pool application system to create a particle effect
4     // Set the state's current scene's clear color to black
5     GetScene()->SetClearColor(CS::Colour::k_black);
6
7     // Retrieve an application system that will help create basic entities like
8     // cameras, rooms, and lights.
9     auto basicEntityFactory = CS::Application::Get()
10         ->GetSystem<Common::BasicEntityFactory>();
11
12     // Create a basic room and add it to the scene
13     CS::EntitySPtr room = basicEntityFactory->CreateRoom();
14     room->GetTransform().SetPosition(0.0f, 10.0f, 0.0f);
15     GetScene()->Add(room);
16
17     // Create a third person camera and add it to the scene
18     auto camera = basicEntityFactory
19         ->CreateThirdPersonCamera(room, CS::Vector3(0.0f, -9.0f, 0.0f));

```

```
20     GetScene()->Add(std::move(camera));
21
22     // Create an ambient light and add it to the scene
23     CS::EntitySPtr ambientLight = basicEntityFactory
24                                   ->CreateAmbientLight
25                                   (CS::Colour(0.65f, 0.65f, 0.65f, 1.0f));
26     GetScene()->Add(ambientLight);
27
28     // Create a particle effect entity and add it to the scene
29     auto particleEntity = CreateParticleEffect();
30     GetScene()->Add(std::move(particleEntity));
31 }
```

2.3.3 Architecture

2.3.4 Attributes

2.3.5 Life Cycle

2.3.5.1 Overview

2.3.5.2 Update Event

2.3.5.3 Render Event

CHAPTER 3 THE CHILLISOURCE AUTOMATED PONG GAME

**CHAPTER 4 CHILLISOURCE PARTICLE EFFECT COMPONENT
CASE STUDIES**

CHAPTER 5 CHILLISOURCE PARTICLE EFFECT COMPONENT OBSERVATIONS

BIBLIOGRAPHY

- [1] W. T. Reeves, “Particle systems – a technique for modeling a class of fuzzy objects,” in *Proceedings of the 10th Annual Conference on Computer Graphics and Interactive Techniques*, ser. SIGGRAPH ’83. New York, NY, USA: ACM, 1983, pp. 359–375. [Online]. Available: <http://doi.acm.org/10.1145/800059.801167>
- [2] ChilliSource. (2014) Features. [Online]. Available: <http://www.chillisourceengine.com/features>
- [3] ——. (2014) Life cycle events. [Online]. Available: <http://www.chillisourceengine.com/life-cycle-events>
- [4] ——. (2014) Basic structure. [Online]. Available: <http://www.chillisourceengine.com/basic-structure>
- [5] Halixi72. (2007) Particle emitter. [Online]. Available: https://en.wikipedia.org/wiki/File:Particle_Emitter.jpg
- [6] ——. (2007) Strand emitter. [Online]. Available: https://en.wikipedia.org/wiki/File:Strand_Emitter.jpg
- [7] ChilliSource. (2016) Chillisource v1.6.0 code repository. [Online]. Available: <https://bitbucket.org/Chilli-Ian/chillisource-ag/src/781988bb9a8ee518c4ea6545ac0e5c55238ec562?at=temp/particleMetrics>